Module-6- Python Fundamentals

1. Introduction to Python:

Q.1. Introduction to Python and its Features (simple, high-level, interpreted language).

Ans: Introduction to Python and its Features

Python is a simple and easy programming language that is widely used for software development, web development, data analysis, automation, and many other applications. It was created by **Guido van Rossum** and first released in **1991**. Python is designed so that we can write programs in a clear and readable way, which makes it suitable for beginners as well as professionals.

One of the main advantages of Python is its **simple syntax**. The code looks very similar to normal English sentences, so we can understand and write programs easily.

Example:

```
print("Hello World")
```

we simply use the print() function to display the message **Hello World** on the screen.

Features of Python

1. Simple Language
Python is easy to learn and easy to write. The syntax is clean and readable, which helps us understand the program quickly.

Example:

```
a = 10
b = 20
print(a + b)
```

This program simply adds two numbers and prints the result.

2. High-Level Language
Python is a high-level language, which means we do not need to worry about complex details like memory management or hardware operations. Python handles these things automatically.

Example:

```
name = "Mithlesh"
print("Hello", name)
```

Here we directly use variables and print the output without worrying about system-level details.

3. Interpreted Language

Python is an interpreted language. This means the code is executed **line by line** by the Python interpreter. We do not need to compile the program before running it.

Example:

```
x = 5
y = 3
print(x * y)
```

The interpreter reads each line and executes it immediately.

4. Portable

Python programs can run on different operating systems such as Windows, Linux, and macOS without major changes.

5. Large Library Support

Python provides many built-in libraries that help us perform tasks like web development, data analysis, and automation easily.

Conclusion

Python is a powerful and beginner-friendly programming language. Because of its **simple syntax, high-level nature, interpreted execution, and large library support**, we can use Python for many types of applications efficiently.

Q.2. History and evolution of Python.

Ans: **History and Evolution of Python**

Python is a popular programming language that was created by **Guido van Rossum** in the late 1980s and officially released in **1991**. It was developed at the **Centrum Wiskunde & Informatica (CWI)** in the Netherlands. The main goal of Python was to create a programming language that is simple, readable, and easy for programmers to use.

The name **Python** was inspired by the comedy television show **"Monty Python's Flying Circus"**, not from the snake. Guido van Rossum wanted a short and unique name for the language.

Early Development

In **1991**, the first version **Python 0.9.0** was released. This version already included many important features such as variables, loops, functions, and basic data types like lists and strings.

Example:

```
a = 5
b = 10
print(a + b)
```

In this example, we simply add two numbers and display the result.

Python 2

In **2000**, **Python 2.0** was released. This version introduced many new features such as list comprehensions, better memory management, and support for Unicode. Python 2 became very popular and was widely used for many years.

Example of list comprehension:

```
numbers = [1,2,3,4]
square = [n*n for n in numbers]
print(square)
```

Python 3

In **2008**, **Python 3.0** was introduced. This version improved the language design and removed some older features of Python 2. Python 3 provides better performance, improved Unicode support, and more consistent syntax.

Example:

```
name = "Mithlesh"
print("Hello", name)
```

Today, Python 3 is the most widely used version and Python 2 is no longer supported.

Conclusion

Python has evolved from a simple scripting language to one of the most powerful programming languages in the world. Because of its **simplicity, readability, and large community support**, we use Python in many fields such as **web development, artificial intelligence, data science, automation, and software development**.

Q.3. Advantages of using Python over other programming languages.

Ans: Python is one of the most popular programming languages in the world. Many developers prefer Python because it is simple, powerful, and easy to learn. It is widely used in web development, data science, automation, artificial intelligence, and software development.

1. Easy to Learn and Use

Python has a very simple and readable syntax. We can understand the code easily compared to many other programming languages like C or Java.

Example:

```
print("Hello World")
```

This single line prints a message on the screen.

2. Less Code Required

In Python, we can write programs with fewer lines of code. This saves time and makes development faster.

Example:

```
a = 10
b = 20
print(a + b)
```

Here we simply add two numbers using only a few lines.

3. Interpreted Language

Python is an interpreted language, which means the code runs line by line. We do not need to compile the program before running it. This makes testing and debugging easier.

Example:

```
x = 5
y = 2
print(x * y)
```

The interpreter reads and executes each line directly.

4. Large Standard Library

Python provides many built-in libraries and modules. These libraries help us perform complex tasks such as web development, data analysis, and automation without writing everything from scratch.

Example:

```
import math
print(math.sqrt(16))
```

Here we use the **math** library to find the square root of a number.

5. Platform Independent

Python programs can run on different operating systems like Windows, Linux, and macOS without major changes. This makes Python highly portable.

6. Strong Community Support

Python has a large community of developers. If we face any problem while programming, we can easily find solutions, tutorials, and documentation online.

Conclusion

Python has many advantages such as **simple syntax, less coding, easy debugging, powerful libraries, and platform independence**. Because of these features, Python is widely used and preferred over many other programming languages.

Q.4. Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

Ans: Python is easy to install and use for programming. To start working with Python, we need to install Python and choose a development environment (IDE) where we can write and run our programs. Common tools used for Python development are **Anaconda, PyCharm, and Visual Studio Code (VS Code)**.

1. Installing Python

First, we need to install Python on our computer.

Steps to install Python:

1. Go to the official Python website: https://www.python.org
2. Click on **Download Python**.
3. Run the downloaded installer.
4. Select the option **"Add Python to PATH"**.
5. Click **Install Now** and complete the installation.

After installation, we can check whether Python is installed correctly.

Example command in terminal or command prompt:

```
python --version
```

This command shows the installed Python version.

2. Using Anaconda

Anaconda is a Python distribution mainly used for data science, machine learning, and scientific computing. It includes Python along with many useful libraries and tools.

Steps to install Anaconda:

1. Visit https://www.anaconda.com
2. Download the Anaconda installer for Windows, Linux, or macOS.
3. Run the installer and follow the instructions.
4. After installation, open **Anaconda Navigator** to manage Python environments and packages.

Example Python program we can run in Anaconda:

```
print("Python is running through Anaconda")
```

3. Using PyCharm

PyCharm is a popular IDE for Python development. It provides features like code suggestions, debugging tools, and project management.

Steps to install PyCharm:

1. Visit https://www.jetbrains.com/pycharm
2. Download **PyCharm Community Edition** (free).
3. Install and open the software.
4. Create a new project and select the installed Python interpreter.

Example program in PyCharm:

```
a = 10
    b = 5
    print(a + b)
```

4. Using Visual Studio Code (VS Code)

VS Code is a lightweight and powerful code editor that supports many programming languages including Python.

Steps to set up Python in VS Code:

1. Download VS Code from https://code.visualstudio.com
2. Install and open the editor.
3. Go to **Extensions** and install the **Python extension**.
4. Select the Python interpreter from the command palette.

Example program in VS Code:

```
name = "Python"
    print("Learning", name)
```

Conclusion

To start programming in Python, we first install Python and then choose a development environment such as **Anaconda, PyCharm, or VS Code**. These tools help us write, run, and manage Python programs easily and efficiently.

Q.5. Writing and executing your first Python program.

Ans: Python is easy to start with because we can write and run a program in a few simple steps. The first program beginners usually write is the **"Hello World"** program.

Step 1: Write the Program

Open a Python editor such as **IDLE, VS Code, or PyCharm** and type the following code:

```
print("Hello World")
```

The print() function is used to display output on the screen.

Step 2: Save the File

Save the file with the **.py extension**, for example:

hello.py

Step 3: Run the Program

Open the terminal or command prompt and run the program using:

```
python hello.py
```

Output

Hello World

Example

```
name = "Mithlesh"
        print("My name is", name)
```

Output

My name is Mithlesh

This is how we write and execute our first Python program.

2. Programming Style:

Q.1. Understanding Python's PEP 8 guidelines

Ans: PEP 8 is the **official style guide for Python code**. It provides rules and recommendations for writing clean, readable, and consistent Python programs. By following PEP 8, we can make our code easier for others to understand and maintain.

1. Proper Indentation

Python recommends using **4 spaces for indentation**. Indentation helps organize the code structure.

Example:

```
if 5 > 2:
  print("Five is greater than two")
```

2. Naming Conventions

Variable and function names should be written in **lowercase with underscores**.

Example:

```
total_marks = 90
student_name = "Mithlesh"
```

3. Spacing Around Operators

Spaces should be used around operators to improve readability.

Example:

```
a = 10
b = 5
sum = a + b
print(sum)
```

4. Line Length

PEP 8 recommends that each line should not be more than **79 characters** to keep the code easy to read.

Conclusion

PEP 8 guidelines help us write **clean, readable, and well-organized Python code**. By following these rules, our programs become easier to understand and maintain.

Q.2. Indentation, comments, and naming conventions in Python.

Ans: In Python, writing clean and organized code is very important. Three important practices that help us write better programs are **indentation, comments, and naming conventions**.

1. Indentation

Indentation means leaving spaces at the beginning of a line to show the structure of the program. In Python, indentation is necessary because it defines blocks of code. Usually, we use **4 spaces** for indentation.

Example:

```
if 10 > 5:
  print("10 is greater than 5")
```

Here, the print() statement is indented to show that it belongs to the **if statement block**.

2. Comments

Comments are used to explain the code. They are not executed by Python and help others understand the program. In Python, comments start with the **# symbol**.

Example:

```
# This program prints a message
print("Welcome to Python")
```

3. Naming Conventions

Naming conventions are rules for giving meaningful names to variables and functions. In Python, names are usually written in **lowercase with underscores**.

Example:

```
student_name = "Mithlesh"
total_marks = 85
print(student_name, total_marks)
```

Conclusion

Indentation helps organize the code structure, comments explain the program, and proper naming conventions make the code easy to read and understand. Following these practices helps us write clean and maintainable Python programs.

Q.3. Writing readable and maintainable code.

Ans: Readable and maintainable code means writing programs in a way that is easy to understand and modify later. When code is clear and well organized, we can easily find errors, update features, and work with other developers.

1. Use Meaningful Variable Names

Variables should have clear and descriptive names so we can easily understand their purpose.

Example:

```
student_name = "Mithlesh"
total_marks = 90
print(student_name, total_marks)
```

2. Use Proper Indentation

Indentation helps organize the code and shows which statements belong together. In Python, we usually use **4 spaces** for indentation.

Example:

```
if total_marks > 50:
   print("Student passed")
```

3. Add Comments

Comments explain what the code is doing. This makes the program easier to understand for others.

Example:

```
# This program calculates the sum of two numbers
a = 10
b = 20
print(a + b)
```

4. Keep Code Simple

Programs should be written in a simple and clear way instead of making them unnecessarily complex.

Example:

```
num = 5
print(num * num)
```

Conclusion

Readable and maintainable code uses **clear variable names, proper indentation, helpful comments, and simple logic**. These practices help us understand, modify, and maintain the program easily in the future.

3. Core Python Concepts:

Q.1. Understanding data types: integers, floats,strings, lists, tuples, dictionaries,sets.

Ans: Data types define the type of value stored in a variable. Python provides different data types to store numbers, text, and collections of data.

1. Integer

An **integer** is a whole number without a decimal point.

Example:

```
a = 10
b = 5
print(a + b)
```

Output: 15

2. Float

A **float** is a number that contains a decimal point.

Example:

```
price = 99.5
print(price)
```

Output: 99.5

3. String

A **string** is a sequence of characters used to store text. Strings are written inside quotes.

Example:

```
name = "Mithlesh"
print(name)
```

Output: Mithlesh

4. List

A **list** is a collection of items stored in square brackets []. Lists can be changed (mutable).

Example:

```
fruits = ["apple", "banana", "mango"]
print(fruits)
```

Output: ['apple', 'banana', 'mango']

5. Tuple

A **tuple** is similar to a list but cannot be changed after creation. Tuples use parentheses ().

Example:

```
numbers = (1, 2, 3)
print(numbers)
```

Output: (1, 2, 3)

6. Dictionary

A **dictionary** stores data in **key–value pairs** using curly brackets {}.

Example:

```
student = {"name": "Mithlesh", "age": 25}
print(student)
```

Output: {'name': 'Mithlesh', 'age': 25}

7. Set

A **set** is a collection of unique elements and does not allow duplicates.

Example:

```
numbers = {1, 2, 3, 3, 4}
print(numbers)
```

Output: {1, 2, 3, 4}

Conclusion

Python provides different data types such as **integers, floats, strings, lists, tuples, dictionaries, and sets** to store and manage different kinds of data efficiently.

Q.2. Python variables and memory allocation.

Ans: A **variable** in Python is used to store data or values in a program. When we create a variable, Python automatically allocates memory to store that value. We do not need to manually manage memory because Python handles it automatically.

Variables in Python

In Python, a variable is created when we assign a value using the **assignment operator (=)**.

Example:

```
a = 10
name = "Mithlesh"
print(a)
print(name)
```

Here, a stores the value **10** and name stores the text **Mithlesh**.

Memory Allocation

When we assign a value to a variable, Python creates an object in memory and stores the value there. The variable name simply refers to that memory location.

Example:

```
x = 20
y = x
print(x)
print(y)
```

In this example, the value **20** is stored in memory, and both x and y refer to that value.

Conclusion

Variables are used to store data in Python programs. When we assign a value to a variable, Python automatically allocates memory and manages it internally, which makes programming easier and more efficient.

Q.3. Python operators: arithmetic, comparison, logical, bitwise.

Ans: Operators in Python are symbols used to perform operations on variables and values. They help us perform calculations, comparisons, and logical operations in programs.

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, and division.

Example:

```
a = 10
b = 5
print(a + b)   # Addition
print(a - b)   # Subtraction
print(a * b)   # Multiplication
print(a / b)   # Division
```

Output: 15, 5, 50, 2.0

2. Comparison Operators

Comparison operators are used to compare two values. The result is either **True** or **False**.

Example:

```
a = 10
b = 5
print(a > b)
print(a == b)
```

Output: True, False

3. Logical Operators

Logical operators are used to combine multiple conditions.

Example:

```
a = 10
b = 5
print(a > 5 and b < 10)
```

Output: True

Common logical operators are **and, or, and not**.

4. Bitwise Operators

Bitwise operators perform operations on binary numbers.

Example:

```
a = 5
b = 3
print(a & b)
```

Output: 1

Here & performs a **bitwise AND operation** on the binary values of the numbers.

Conclusion

Python operators such as **arithmetic, comparison, logical, and bitwise** help us perform calculations, compare values, and apply logical operations in Python programs.

4. Conditional Statements:

Q.1. Introduction to conditionalstatements: if, else, elif

Ans: Conditional statements in Python are used to make decisions in a program. They allow the program to execute different blocks of code depending on whether a condition is **True or False**.

1. if Statement

The **if statement** is used to check a condition. If the condition is true, the code inside the if block runs.

Example:

```
age = 18
if age >= 18:
  print("Eligible to vote")
```

If the condition age >= 18 is true, the message will be printed.

2. else Statement

The **else statement** is used when the condition in the if statement is false.

Example:

```
age = 16
if age >= 18:
  print("Eligible to vote")
else:
  print("Not eligible to vote")
```

If the condition is false, the code inside the **else** block runs.

3. elif Statement

The **elif (else if)** statement is used to check multiple conditions.

Example:

```
marks = 75

if marks >= 90:
  print("Grade A")
elif marks >= 60:
  print("Grade B")
```

```
else:
  print("Grade C")
```

Conclusion

Conditional statements such as **if, elif, and else** help us control the flow of a program by executing different code blocks based on conditions.

Q.2. Nested if-else conditions

Ans: A **nested if-else** means using an **if-else statement inside another if-else statement**. It is used when we need to check multiple conditions step by step.

Example-1:

```
age = 20
citizen = True

if age >= 18:
  if citizen:
    print("Eligible to vote")
  else:
    print("Not a citizen")
else:
  print("Underage")
```

In this example, the first **if** checks the age. If the age condition is true, another **if** inside it checks whether the person is a citizen.

Example-2:

```
num = 10

if num > 0:
  if num % 2 == 0:
    print("Positive even number")
  else:
    print("Positive odd number")
else:
  print("Negative number")
```

Conclusion

Nested if-else conditions help us check **multiple related conditions** inside a program and make more detailed decisions based on different situations.

5. Looping (For, While):

Q.1. Introduction to for and while loops.

Ans: Loops in Python are used to **repeat a block of code multiple times**. They help us avoid writing the same code again and again. The two main loops in Python are **for loop** and **while loop**.

1. for Loop

A **for loop** is used when we know how many times the loop should run. It is commonly used to iterate over a sequence like a list, string, or range.

Example:

```
for i in range(5):
  print(i)
```

Output:

```
0
1
2
3
4
```

Here the loop runs **5 times** and prints numbers from 0 to 4.

2. while Loop

A **while loop** runs as long as a given condition is **True**. It stops when the condition becomes **False**.

Example:

```
i = 1
while i <= 5:
  print(i)
  i = i + 1
```

Output:

```
1
2
3
4
5
```

In this example, the loop continues until the value of **i** becomes greater than 5.

Conclusion

Loops such as **for** and **while** help us repeat tasks efficiently in Python programs. The **for loop** is used when the number of iterations is known, while the **while loop** runs based on a condition.

Q.2. How loops work in Python?

Ans: Loops in Python are used to **repeat a block of code multiple times** until a certain condition is met. They help us perform repetitive tasks easily without writing the same code again and again.

In Python, loops work by **executing a set of statements repeatedly** based on a condition or a sequence of values.

Example using a for Loop

```
for i in range(3):
  print("Learning Python")
```

Output:

```
Learning Python
Learning Python
Learning Python
```

Here, the loop runs **3 times** because range(3) generates numbers from 0 to 2.

Example using a while Loop

```
i = 1
while i <= 3:
  print(i)
  i = i + 1
```

Output:

```
1
2
3
```

In this example, the loop continues running **while the condition i <= 3 is true**. When the condition becomes false, the loop stops.

Conclusion

Loops in Python repeat a block of code either for a **fixed number of times (for loop)** or **until a condition becomes false (while loop)**. This makes programs shorter, easier to write, and more efficient.

Q.3. Using loops with collections (lists, tuples, etc.).

Ans: In Python, loops are often used to **go through each element of a collection** such as lists, tuples, or sets. This process is called **iteration**. By using loops, we can access and work with each item one by one.

Example with a List

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
  print(fruit)
```

Output:

```
apple
banana
mango
```

Here the **for loop** goes through each item in the list and prints it.

Example with a Tuple

```
numbers = (1, 2, 3, 4)

for num in numbers:
  print(num)
```

Output:

```
1
2
3
4
```

In this example, the loop accesses each number in the tuple.

Example with a String

```
name = "Python"

for letter in name:
  print(letter)
```

Output:

```
P
y
t
h
o
n
```

Conclusion

Loops help us easily **iterate through collections like lists, tuples, and strings**. This allows us to process each element in a collection one by one in a simple and efficient way.

8. Control Statements (Break, Continue, Pass):

Q.1. Understanding the role of break, continue, and pass in Python loops.

Ans: In Python loops, the keywords **break**, **continue**, and **pass** are used to control how the loop behaves during execution.

1. break Statement

The **break** statement is used to **stop the loop immediately**, even if the loop condition is still true.

Example:

```
for i in range(5):
  if i == 3:
    break
  print(i)
```

Output:

```
0
1
2
```

Here the loop stops when i becomes **3**.

2. continue Statement

The **continue** statement is used to **skip the current iteration** and move to the next iteration of the loop.

Example:

```
for i in range(5):
  if i == 2:
    continue
  print(i)
```

Output:

```
0
1
```

3
4

Here the value **2** is skipped.

3. pass Statement

The **pass** statement is used when we need a placeholder but do not want the loop to perform any action.

Example:

```
for i in range(3):
  pass

print("Loop finished")
```

Output:

Loop finished

Conclusion

The **break** statement stops the loop, **continue** skips the current iteration, and **pass** acts as a placeholder when no action is required in the loop. These statements help control the flow of loops in Python.

9. String Manipulation:

Q.1. Understanding how to access and manipulate strings.

Ans: A **string** in Python is a sequence of characters used to store text. We can **access characters** in a string and also **modify or manipulate the text** using different methods.

1. Accessing Characters in a String

Characters in a string can be accessed using **index numbers**. Indexing starts from **0**.

Example:

```
name = "Python"
print(name[0])
print(name[3])
```

Output:

P
h

Here name[0] gives the first character and name[3] gives the fourth character.

2. String Slicing

We can extract a part of a string using **slicing**.

Example:

```
text = "Programming"
print(text[0:6])
```

Output:

Progra

This selects characters from index **0 to 5**.

3. String Manipulation

Python provides methods to modify or process strings.

Example:

```
message = "hello python"
print(message.upper())
```

Output:

HELLO PYTHON

Another example:

```
word = "Python"
print(len(word))
```

Output:

6

Conclusion

Strings in Python can be **accessed using indexing**, **extracted using slicing**, and **modified using built-in functions and methods**. These features help us easily work with text in Python programs.

Q.2. Basic operations: concatenation, repetition,string methods(upper(), lower(), etc.).

Ans: In Python, strings can be combined, repeated, and modified using different operations and built-in methods.

1. String Concatenation

Concatenation means joining two or more strings together using the **+ operator**.

Example:

```
first = "Hello"
second = "World"
result = first + " " + second
print(result)
```

Output:

Hello World

Here two strings are joined to form one string.

2. String Repetition

We can repeat a string multiple times using the *** operator**.

Example:

```
word = "Python "
print(word * 3)
```

Output:

Python Python Python

3. String Methods

Python provides many built-in methods to modify strings.

Example of **upper()** and **lower()**:

```
text = "Python Programming"

print(text.upper())
print(text.lower())
```

Output:

PYTHON PROGRAMMING
python programming

Another example:

```
name = "mithlesh"
print(name.capitalize())
```

Output:

Mithlesh

Conclusion

In Python, we can perform basic string operations such as **concatenation using +, repetition using *, and modifying strings using methods like upper(), lower(), and capitalize()**. These operations help us work easily with text data.

Q.3. String slicing.

Ans: **String slicing** in Python is used to extract a part of a string. It allows us to get a specific portion of characters from a string by using index positions.

The basic syntax of slicing is:

string[start:end]

Here **start** is the beginning index and **end** is the ending index (not included).

Example

```
text = "Python Programming"
print(text[0:6])
```

Output:

Python

This extracts characters from index **0 to 5**.

Example with Different Index

```
word = "Computer"
print(word[1:5])
```

Output:

ompu

Here slicing starts from index **1** and ends before index **5**.

Example with Step

```
name = "Python"
print(name[0:6:2])
```

Output:

Pto

This selects characters by skipping every second character.

Conclusion

String slicing helps us **extract parts of a string using index positions**. It is very useful when we need only a specific section of text from a string.